

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-aa094c034633d6be9.jsonl`
- SHA-256: `815319dae1cc0fa96ed16ae31ba68a214683243e867a00f481dce2bf20795e59`
- Source modified: `2026-06-13T19:12:02+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_095035a0-e9f`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-13T19:09:22.946Z)

Scrutinize the NO-REGRESSION claim. trajectory_hybrid.py was restructured: per-window stats now come from a new `\_enrich\_candidate\_stats` hook (base returns the 4 motion keys), computed into a `window\_stats` list used for BOTH persistence and a new `\_select\_for\_handoff` gate. Verify wasb_hybrid/tracknet_hybrid behaviour is BYTE-IDENTICAL to before: persisted stats unchanged, handoff sends ALL windows, Phase-3/Phase-4 unchanged, the empty-windows guard intact, window_stats computed even when store_candidates=False causes no behavioural change. Also verify FusionSegmenter with DEFAULT config (person OFF, min_crossings=0) seeds + hands off identically to its wasb substrate.

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:09:25.651Z)

I'll scrutinize the NO-REGRESSION claim by reading the actual files and diffs. Let me start by pulling the latest and examining the changed files.

3. user (2026-06-13T19:09:27.368Z)

warning: in the working copy of 'backend/pipeline/segmenters/trajectory_hybrid.py', LF will be replaced by CRLF the next time Git touches it

```
diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
index b2847c9..149282e 100644
```

```
--- a/backend/pipeline/segmenters/trajectory_hybrid.py
+++ b/backend/pipeline/segmenters/trajectory_hybrid.py
@@ -78,6 +78,31 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
     except (ValueError, TypeError):
         return max(3, int(duration / 0.8))
```

```
+ # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged) -----
+ def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
+                             frame_width: float, video_path: str,
+                             idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
```

```

+     """Stats dict persisted into ``candidate_segments.stats`` for one window. The BASE
+     returns exactly the 4 motion keys, so the trajectory twins persist byte-identical
+     rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
+     features). ``points`` are the whole-video trajectory points (TrajectoryPoint)."""
+     return {
+         "peak_velocity": cw["peak_velocity"],
+         "mean_velocity": cw["mean_velocity"],
+         "active_frame_count": cw["active_frame_count"],
+         "track_id_count": cw["track_id_count"],
+     }
+
+ def _select_for_handoff(self, windows: List[Dict[str, Any]],
+                         window_stats: List[Dict[str, Any]],
+                         idx_cfg: Dict[str, Any]) -> List[int]:
+     """Indices of the candidate windows that proceed to the AI handoff (and thus may
+     become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
+     ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
+     Persisted candidates are NOT affected ? they remain the full superset for offline
+     re-scoring."""
+     return list(range(len(windows)))
+
+ def process_video(self, video_id: str, video_path: str, # type: ignore[override]
+                  player_pool: Optional[List[str]] = None,
+                  max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
@@ -172,31 +197,36 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
+         f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
+         f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")
+
+     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the fusion
+     # segmenter adds net_crossings + person-cue features. Computed once, reused for both
+     # persistence and the handoff gate below.
+     window_stats = [
+         self._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
+         for cw in all_candidate_windows
+     ]
+
+     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
+     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
+     if store_candidates:
+         for i, cw in enumerate(all_candidate_windows):
+             stats = {
+                 "peak_velocity": cw["peak_velocity"],
+                 "mean_velocity": cw["mean_velocity"],
+                 "active_frame_count": cw["active_frame_count"],
+                 "track_id_count": cw["track_id_count"],
+             }
+             candidate_ids[i] = self.db.add_candidate_segment(
+                 video_id, detector=self.name,
+                 start_time=cw["start"], end_time=cw["end"],
+                 chunk_index=cw["chunk_index"], confidence=min(1.0, cw["peak_velocity"]),
+                 stats=stats, detection_params=detection_params,
+                 stats=window_stats[i], detection_params=detection_params,
+             )
+
+     if not all_candidate_windows:
+         return []
+
+     # --- Phase 3: AI provider handoff over padded candidate clips ---
+     # Which windows reach the handoff (base = all; fusion may apply Gate A/B). Persisted
+     # candidates above stay the full superset ? only the served play_segments are gated.
+     handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
+     handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")

```

```

        os.makedirs(handoff_dir, exist_ok=True)
        logger.info("Phase 3: %s validation on %d candidate windows...",
-            provider_name, len(all_candidate_windows))
+            provider_name, len(handoff_indices))

    def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
        w_start = max(0, window["start"] - handoff_padding)
@@ -225,8 +255,8 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):

        ai_segments: List[dict] = []
        with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as executor:
-            futures = [executor.submit(process_candidate_window, wi, cw)
-                        for wi, cw in enumerate(all_candidate_windows)]
+            futures = [executor.submit(process_candidate_window, wi, all_candidate_windows[wi])
+                        for wi in handoff_indices]
        for future in futures:
            ai_segments.extend(future.result())

    try:

```

4. user (2026-06-13T19:09:28.362Z)

```
diff --git a/backend/config/models.py b/backend/config/models.py
```

```
index bb56881..7f8191c 100644
```

```
--- a/backend/config/models.py
```

```
+++ b/backend/config/models.py
```

```
@@ -75,6 +75,39 @@ class WasbIndexConfig(BaseModel):
```

```
    min_free_gb: float = 0.0
```

```
+class FusionConfig(BaseModel):
```

```
+    """Config for the multi-signal `fusion_hybrid` segmenter (MULTI-SIGNAL-FUSION-PLAN P2).
```

```
+
+    Every default reproduces control behaviour: the fusion segmenter persists the
+    shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']`` is
+    true ? the person-cue features, but it does NOT gate the served handoff unless a
+    threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court mask
+    is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-only);
+    supplied ? off-court persons (spectators / adjacent courts) are excluded.
+    """
```

```
+    # Which trajectory substrate seeds the windows (shuttle stays primary).
```

```
+    substrate: Literal["wasb", "tracknet"] = "wasb"
```

```
+    # Windowing velocity threshold for the fusion family (the engine reads
```

```
+    # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
```

```
+    # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
```

```
+    velocity_threshold: float = 0.02
```

```
+    # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
```

```
+    # enabled ? so the default path never imports torch / touches the GPU.
```

```
+    cue_flags: dict[str, bool] = Field(default_factory=dict)
```

```
+    # Served Gate A: drop windows whose shuttle net-crossings < this from the AI handoff.
```

```
+    # 0 = OFF (no gating; persisted candidates are always the full superset for offline re-
+    scoring).
```

```
+    min_crossings: int = Field(default=0, ge=0)
```

```
+    # Served Gate B: when the person cue is on, drop windows with median in-court players
```

```
+    # < this. 0 = OFF.
```

```
+    min_players: int = Field(default=0, ge=0)
```

```
+    # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
```

```
+    court_polygon: Optional[list[list[float]]] = None
```

```
+    # Net line for the person two-sided-motion split (person features only ? the shuttle
```

```
+    # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
```

```
+    net_axis: Literal["x", "y"] = "y"
```

```
+    net_position: float = Field(default=0.5, ge=0.0, le=1.0)
```

```
+
+

```

```
class IndexingConfig(BaseModel):
```

```
    default_segmenter: str = "yolo_hybrid"
```

```

        use_gpu: bool = True
@@ -106,6 +139,7 @@ class IndexingConfig(BaseModel):

    tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
    wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
+    fusion: FusionConfig = Field(default_factory=FusionConfig)

    @field_validator("default_segmenter")
    @classmethod
diff --git a/backend/pipeline/detectors/person_detector.py
b/backend/pipeline/detectors/person_detector.py
index 38ec2b3..5b931a4 100644
--- a/backend/pipeline/detectors/person_detector.py
+++ b/backend/pipeline/detectors/person_detector.py
@@ -252,3 +252,50 @@ class PersonDetector:
    windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

    return windows
+
+ def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
+                          frame_skip: int = 3) -> Dict[str, Any]:
+     """Per-frame person detections over `[start_frame, end_frame]` of the ORIGINAL video.
+
+     Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
+     ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and discards the
+     per-frame boxes), this returns the raw per-frame detections so the fusion features can
+     align them with the shuttle trajectory. It reads the source by **absolute frame index**
+     (seek + sequential read), NOT a re-extracted clip, so the returned frame indices match
+     the WASB trajectory's ``.frame`` exactly ? that alignment is what
+     ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
+
+     Returns ``{"frames": {frame_idx: [(track_id, bbox), ...], "fps", "frame_width",
+     "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-seek
+     accuracy varies by codec (it lands near the requested keyframe then decodes forward);
+     treat the indices as ±a few frames on real footage ? fine for window-level features.
+     """
+     self.lazy_load_model()
+     cap = cv2.VideoCapture(video_path)
+     if not cap.isOpened():
+         logger.error(f"Could not open {video_path}")
+         return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
+
+     fps = cap.get(cv2.CAP_PROP_FPS)
+     if fps <= 0:
+         fps = 30.0
+     frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
+     frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
+     score_thresh = self.config.get("indexing", {}).get("detector_score_threshold", 0.5)
+     step = max(frame_skip, 1)
+     tracker = self.make_tracker(fps / step)
+
+     start_frame = max(0, int(start_frame))
+     frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
+     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
+     frame_idx = start_frame
+     while frame_idx <= end_frame:
+         ret, frame = cap.read()
+         if not ret:
+             break
+         if (frame_idx - start_frame) % step == 0:
+             frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
+         frame_idx += 1
+     cap.release()
+     return {"frames": frames, "fps": fps,
+           "frame_width": float(frame_width), "frame_height": float(frame_height)}

```

```

diff --git a/backend/pipeline/segmenters/__init__.py b/backend/pipeline/segmenters/__init__.py
index c003d1a..fc5de77 100644
--- a/backend/pipeline/segmenters/__init__.py
+++ b/backend/pipeline/segmenters/__init__.py
@@ -4,6 +4,7 @@ from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
+from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter

__all__ = [
    "segmenter_registry",
@@ -13,4 +14,5 @@ __all__ = [
    "HybridYoloSegmenter",
    "TrackNetHybridSegmenter",
    "WasbHybridSegmenter",
+    "FusionSegmenter",
]

```

5. assistant (2026-06-13T19:09:30.983Z)

Now let me read the full new files and the surrounding context of trajectory_hybrid.py to verify the NO-REGRESSION claim thoroughly.

6. user (2026-06-13T19:09:32.428Z)

```

1      """Shared engine for trajectory-substrate hybrid segmenters.
2
3      tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4      standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5      now the single home for the whole pipeline shape:
6
7      healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8      -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10     A concrete subclass supplies only its identity and runner wiring:
11     - ``family``          ? config section under ``indexing.*``, output subdir, and
12                           the ``<family>-hybrid-<model>`` detector tag in metadata.
13     - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14     - ``name`` / ``description`` properties (the registry contract).
15     - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16                                     detection_params extras such as weights_path).
17
18     This base is deliberately NOT registered; only concrete subclasses register.
19     """
20     import os
21     import time
22     import logging
23     import concurrent.futures
24     from typing import Any, Dict, List, Optional, Tuple
25
26     import cv2
27
28     from backend.pipeline.segmenters.base import BasePlaySegmenter
29     from backend.utils.video import get_video_duration, extract_video_chunk
30     from backend.sports import sport_registry
31     from backend.providers import provider_registry
32     from backend.pipeline.detectors.base import DetectorRunner
33
34     logger = logging.getLogger(__name__)
35
36
37     def _fmt_ts(seconds: float) -> str:
38         seconds = max(0, int(seconds))
39         h, rem = divmod(seconds, 3600)

```

```

40         m, s = divmod(rem, 60)
41         return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44     class TrajectoryHybridSegmenter(BasePlaySegmenter):
45         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46         boundaries."""
47
48         #: config section under `indexing` (velocity_threshold lives there), the
49         #: output subdir for trajectories, and the metadata detector-tag prefix.
50         family: str = ""
51         #: human-readable label used in log lines.
52         display_label: str = ""
53
54     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
55     Any]]:
56         """Return (runner, detector-specific detection_params extras)."""
57         raise NotImplementedError
58
59     @staticmethod
60     def _probe_fps_width(video_path: str) -> tuple:
61         """Return (fps, frame_width) for a video, with sensible fallbacks."""
62         cap = cv2.VideoCapture(video_path)
63         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
64         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
65         cap.release()
66         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
67
68     @staticmethod
69     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
70         """Provider-reported exchange count, else a duration-based estimate.
71
72         One canonical implementation ? the twins had drifted (wasb relied on
73         ``int(None)`` raising into the fallback; same result, by accident).
74         """
75         shot_count = segment.get("shuttle_exchange_count")
76         if shot_count is None:
77             return max(3, int(duration / 0.8))
78         try:
79             return int(shot_count)
80         except (ValueError, TypeError):
81             return max(3, int(duration / 0.8))
82
83     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
84     -----
85     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
86     frame_width: float, video_path: str,
87     idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
88         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
89         BASE
90         returns exactly the 4 motion keys, so the trajectory twins persist byte-
91         identical
92         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
93         features). ``points`` are the whole-video trajectory points
94         (TrajectoryPoint)."""
95         return {
96             "peak_velocity": cw["peak_velocity"],
97             "mean_velocity": cw["mean_velocity"],
98             "active_frame_count": cw["active_frame_count"],
99             "track_id_count": cw["track_id_count"],
100         }
101
102     def _select_for_handoff(self, windows: List[Dict[str, Any]],
103     window_stats: List[Dict[str, Any]],
104     idx_cfg: Dict[str, Any]) -> List[int]:

```

```

99         """Indices of the candidate windows that proceed to the AI handoff (and thus may
100         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
101         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
102         Persisted candidates are NOT affected ? they remain the full superset for
offline
103         re-scoring."""
104         return list(range(len(windows)))
105
106     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
107                     player_pool: Optional[List[str]] = None,
108                     max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
109         # override-ignore: the ABC declares the Result contract (Success|Failure)
110         # but every live segmenter still returns a bare list ? the documented
111         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
112         label = self.display_label
113         # --- Sport profile + AI provider wiring (identical across segmenters) ---
114         sport_name = self.config.get("sport", "badminton")
115         try:
116             sport_profile = sport_registry.get(sport_name)
117         except KeyError as e:
118             logger.error(str(e))
119             return []
120
121         idx_cfg = self.config.get("indexing", {})
122         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
123         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
124
125         api_key_env_var = f"{provider_name.upper()}_API_KEY"
126         api_key = os.environ.get(api_key_env_var)
127         if not api_key:
128             logger.error(
129                 f"{api_key_env_var} environment variable is not set! "
130                 f"To run the {provider_name} handoff, set your API key. "
131                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
132             )
133             return []
134         try:
135             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
136         except KeyError as e:
137             logger.error(str(e))
138             return []
139
140         video_duration = get_video_duration(video_path)
141         if video_duration <= 0:
142             logger.error("Invalid video duration.")
143             return []
144         fps, frame_width = self._probe_fps_width(video_path)
145
146         # --- Runner config + windowing thresholds ---
147         runner, runner_params = self._build_runner(idx_cfg)
148
149         velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
150         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
151         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
152         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
153         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
154         log_candidates = idx_cfg.get("log_yolo_candidates", True)
155         store_candidates = idx_cfg.get("store_yolo_candidates", True)
156
157         detection_params = {
158             "detector": self.name,
159             "velocity_thresh": velocity_thresh,

```

```

160         "merge_gap": merge_gap,
161         "min_action_window_duration": min_window_duration,
162         **runner_params,
163     }
164
165     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
166     ok, msg = runner.healthcheck()
167     if not ok:
168         logger.error("%s WSL environment not ready: %s", label, msg)
169         return []
170     logger.info("%s WSL env OK: %s", label, msg)
171
172     # --- Phase 2: trajectory -> candidate rally windows ---
173     # Trajectory detectors process the whole video in one pass (they carry
174     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
175     t_start = time.time()
176     out_dir = os.path.join("output", self.family)
177     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
178     if not csv_path:
179         logger.error("%s produced no trajectory; aborting.", label)
180         return []
181
182     points = runner.parse_trajectory_csv(csv_path)
183     logger.info("Parsed %d trajectory points (%d visible).",
184               len(points), sum(1 for p in points if p.visible))
185
186     all_candidate_windows = runner.trajectory_to_action_windows(
187         points, fps=fps, frame_width=frame_width, chunk_start=0.0,
188         velocity_thresh=velocity_thresh, merge_gap=merge_gap,
189         min_window_duration=min_window_duration, chunk_index=0,
190     )
191     logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
192               label, time.time() - t_start, len(all_candidate_windows))
193
194     if log_candidates:
195         for cw in all_candidate_windows:
196             logger.info(f"[{label} rally]
197             {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
198             f"({cw['end'] - cw['start']:.1f}s) |
199             frames={cw['active_frame_count']} "
200             f"peak_v={cw['peak_velocity']:.3f}
201             mean_v={cw['mean_velocity']:.3f}")
202
203     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
204     fusion
205     # segmenter adds net_crossings + person-cue features. Computed once, reused for
206     both
207     # persistence and the handoff gate below.
208     window_stats = [
209         self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
210         idx_cfg)
211         for cw in all_candidate_windows
212     ]
213
214     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
215     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
216     if store_candidates:
217         for i, cw in enumerate(all_candidate_windows):
218             candidate_ids[i] = self.db.add_candidate_segment(
219                 video_id, detector=self.name,
220                 start_time=cw["start"], end_time=cw["end"],
221                 chunk_index=cw["chunk_index"], confidence=min(1.0,
222                 cw["peak_velocity"]),
223                 stats=window_stats[i], detection_params=detection_params,
224             )

```



```

218
219     if not all_candidate_windows:
220         return []
221
222     # --- Phase 3: AI provider handoff over padded candidate clips ---
223     # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
224     # candidates above stay the full superset ? only the served play_segments are
gated.
225     handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
226     handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
227     os.makedirs(handoff_dir, exist_ok=True)
228     logger.info("Phase 3: %s validation on %d candidate windows...",
229                 provider_name, len(handoff_indices))
230
231     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
232         w_start = max(0, window["start"] - handoff_padding)
233         w_end = min(video_duration, window["end"] + handoff_padding)
234         w_dur = w_end - w_start
235         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
236         if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
237             return []
238         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
239         segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
240                                             label=f"Handoff {wi+1}")
241         valid = []
242         for seg in segments:
243             try:
244                 seg["start_time"] = float(seg["start_time"]) + w_start
245                 seg["end_time"] = float(seg["end_time"]) + w_start
246                 seg["candidate_id"] = candidate_ids[wi]
247                 valid.append(seg)
248             except (ValueError, TypeError, KeyError) as e:
249                 logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
250             try:
251                 os.remove(w_path)
252             except OSError:
253                 pass
254         return valid
255
256     ai_segments: List[dict] = []
257     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
258         futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
259                     for wi in handoff_indices]
260         for future in futures:
261             ai_segments.extend(future.result())
262         try:
263             os.rmdir(handoff_dir)
264         except OSError:
265             pass
266         logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
267
268     # --- Phase 4: DB population + candidate linking ---
269     segments_data = []
270     for segment in ai_segments:
271         try:
272             start = float(segment["start_time"])
273             end = float(segment["end_time"])
274             dur = end - start

```

```

275         metadata = {
276             "shot_count": self._shot_count_from(segment, dur),
277             "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
278             "detector": f"{self.family}-hybrid-{model_name}",
279         }
280         segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
281         if segment_id:
282             candidate_id = segment.get("_candidate_id")
283             if candidate_id is not None:
284                 self.db.link_candidate_to_segment(candidate_id, segment_id)
285             segments_data.append({
286                 "id": segment_id, "start_time": start, "end_time": end,
287                 "duration": dur, "metadata": metadata,
288             })
289         except Exception as e:
290             logger.warning("Skipping invalid segment struct: %s", e)
291
292     return segments_data
293

```

7. user (2026-06-13T19:09:32.655Z)

```

1     """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3     Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4     TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5     do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6     shuttle-seeded window via two overridable seams the engine exposes:
7
8     - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9     GPU-free, computed from the trajectory we already have); and, only when the person cue
10    is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11    (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12    fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
13    never imports torch / touches the GPU.
14    - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
15    (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
16    dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
17    **Persisted candidates remain the full superset** regardless, so the offline
experiment
18    harness can re-score any variant.
19
20    Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
21    nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
22    and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
23    segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
24    experiment harness (EXPERIMENT_HARNESS.md), never silently.
25    """
26    from typing import Any, Dict, List, Optional, Tuple
27
28    from backend.pipeline.segmenters.base import segmenter_registry
29    from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
30    from backend.pipeline.detectors.base import DetectorRunner
31    from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
32    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
33    from backend.pipeline.detectors import rally_gate
34    from backend.pipeline.detectors import fusion_features as ff

```

```

35
36
37     class FusionSegmenter(TrajectoryHybridSegmenter):
38         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
39         B)."""
40         family = "fusion"
41         display_label = "Fusion"
42
43         def __init__(self, db: Any, config: Any):
44             super().__init__(db, config)
45             # Built lazily only if the person cue is enabled (keeps torch/GPU off the
46             default path).
47             self._person: Optional[Any] = None
48
49         @property
50         def name(self) -> str:
51             return "fusion_hybrid"
52
53         @property
54         def description(self) -> str:
55             return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
56             rallies, "
57             "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
58             and the "
59             "AI provider sets semantic boundaries.")
60
61         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
62             Any]]:
63             substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
64             if substrate == "tracknet":
65                 cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
66                 return TrackNetRunner(cfg), {
67                     "weights_path": cfg.weights_path,
68                     "queue_length": cfg.queue_length,
69                     "fusion_substrate": "tracknet",
70                 }
71             wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
72             return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
73             "wasb"}
74
75         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
76             frame_width: float, video_path: str,
77             idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
78             stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
79             video_path, idx_cfg)
80             # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
81             persisted
82             # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
83             estimated
84             # over the whole trajectory by rally_gate (camera-agnostic).
85             gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])]), fps,
86             min_crossings=0)
87             stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
88
89             fcfg = idx_cfg.get("fusion", {})
90             if fcfg.get("cue_flags", {}).get("person", False):
91                 stats.update(self._person_features(cw, points, fps, frame_width, video_path,
92                 fcfg))
93             return stats
94
95         def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
96             frame_width: float, video_path: str,
97             fcfg: Dict[str, Any]) -> Dict[str, float]:
98             """GPU path (person cue ON): run the person detector over this window's

```

```

ORIGINAL-video
89         frame range and compute the scale/translation-invariant person features. Lazily
builds
90         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
91         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
92         from backend.pipeline.detectors.person_detector import PersonDetector
93         if self._person is None:
94             self._person = PersonDetector(self.config)
95
96         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
97         start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)
98         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
99         fw = det["frame_width"] or frame_width
100        fh = det["frame_height"] or 0.0
101
102        poly = fcfg.get("court_polygon")
103        polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
104        net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
105        shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
106        return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
107                                         court_polygon=polygon, net=net)
108
109        def _select_for_handoff(self, windows: List[Dict[str, Any]],
110                                window_stats: List[Dict[str, Any]],
111                                idx_cfg: Dict[str, Any]) -> List[int]:
112        """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
113        (0) keep every window ? identical to the substrate. Persisted candidates are
114        untouched (full superset)."""
115        fcfg = idx_cfg.get("fusion", {})
116        min_crossings = int(fcfg.get("min_crossings", 0))
117        min_players = int(fcfg.get("min_players", 0))
118        person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120        keep: List[int] = []
121        for i, st in enumerate(window_stats):
122            if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                continue
124            if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                continue
126            keep.append(i)
127        return keep
128
129
130        segmenter_registry.register(FusionSegmenter)
131

```

8. assistant (2026-06-13T19:09:35.263Z)

Now let me read the fusion_features.py and the relevant parts of rally_gate and the existing wasb/tracknet hybrid subclasses to confirm the substrate behavior.

9. user (2026-06-13T19:09:39.529Z)

```

1        """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3        NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4        math is unit-testable in a GPU-free dev container. The detection (person boxes,
5        shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6        module only turns those signals into a handful of scale/translation-invariant
7        features (counts / ratios / zone-membership ? never raw pixel coordinates, which would

```

```

8     memorise this court/camera and fail to generalise off a small labelled set).
9
10    Conventions:
11    - A person box is ``(x1, y1, x2, y2)`` in **pixels**; ``(track_id, box)`` when motion
needs it.
12    - ``per_frame`` is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video
frame
13        index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
timing).
14    - A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
trajectory shape.
15    - ``court_polygon`` is normalised 0-1 ``[(x, y), ...]`` or ``None`` (None ? no mask,
every
16        detection counts ? the player-count-only mode).
17    - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
0.0).
18
19    Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20    floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
21    """
22    from __future__ import annotations
23
24    from typing import Dict, List, Optional, Sequence, Tuple
25
26    BBox = Tuple[float, float, float, float]
27    Net = Tuple[str, float]
28
29
30    def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31        """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32        (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33        if polygon is None or len(polygon) < 3:
34            return False
35        x, y = point
36        inside = False
37        n = len(polygon)
38        j = n - 1
39        for i in range(n):
40            xi, yi = polygon[i]
41            xj, yj = polygon[j]
42            # Does the horizontal ray at y cross edge (i, j)?
43            if (yi > y) != (yj > y):
44                x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45                if x < x_cross:
46                    inside = not inside
47            j = i
48        return inside
49
50
51    def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52        """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53        the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54        if frame_width <= 0 or frame_height <= 0:
55            return (0.0, 0.0)
56        x1, _y1, x2, y2 = box
57        return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60    def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61        if frame_width <= 0 or frame_height <= 0:
62            return (0.0, 0.0)
63        x1, y1, x2, y2 = box

```

```

64         return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67     def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69         """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70         True (count everyone) when it is None."""
71         if court_polygon is None:
72             return True
73         return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
74
75
76     def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
77                       frame_height: float,
78                       court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
79     List[int]:
80         """Per-frame count of in-court persons (one entry per sampled frame)."""
81         return [
82             sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
83 court_polygon))
84             for _frame_idx, dets in sorted(per_frame.items())
85         ]
86
87     def _median(vals: Sequence[float]) -> float:
88         if not vals:
89             return 0.0
90         s = sorted(vals)
91         n = len(s)
92         return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
93
94     def players_in_court(counts: Sequence[int]) -> float:
95         """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
96         return _median(list(counts))
97
98
99     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
100         """Fraction of sampled frames with >= `min_players` in court (track stability vs
101 flicker)."""
102         if not counts:
103             return 0.0
104         return sum(1 for c in counts if c >= min_players) / len(counts)
105
106     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
107                          frame_width: float, frame_height: float) -> float:
108         """Mean per-track centre displacement between consecutive sampled frames, normalised
109         by the frame width (resolution-invariant). 0.0 if <2 frames or no shared tracks."""
110         if frame_width <= 0 or len(per_frame) < 2:
111             return 0.0
112         frames = sorted(per_frame.keys())
113         steps: List[float] = []
114         for a, b in zip(frames, frames[1:]):
115             prev = {tid: box for tid, box in per_frame[a]}
116             for tid, box in per_frame[b]:
117                 if tid in prev:
118                     cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
119                     cx1, cy1 = _center_norm(box, frame_width, frame_height)
120                     steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
121         if not steps:
122             return 0.0
123         return sum(steps) / len(steps)
124
125

```

```

126 def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
127                       frame_height: float, net: Optional[Net],
128                       court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
129     """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
130
131     A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
132     The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
133     0.0.
134     """
135     if net is None or not per_frame:
136         return 0.0
137     axis, pos = net
138     both = 0
139     for _frame_idx, dets in per_frame.items():
140         near, far = False, False
141         for _tid, box in dets:
142             if not person_in_court(box, frame_width, frame_height, court_polygon):
143                 continue
144             fx, fy = _foot_norm(box, frame_width, frame_height)
145             coord = fx if axis == "x" else fy
146             if coord < pos:
147                 near = True
148             else:
149                 far = True
150             if near and far:
151                 both += 1
152     return both / len(per_frame)
153
154 def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
155                    pad_frac: float) -> bool:
156     """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
157     width/height (the 'adjacent / in-hand' tolerance)?"""
158     x1, y1, x2, y2 = box
159     px = pad_frac * frame_width
160     py = pad_frac * frame_height
161     return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
162
163 def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
164                               frame_width: float, frame_height: float,
165                               pad_frac: float = 0.03) -> float:
166     """Fraction of (visible shuttle frames that were also person-sampled) in which the
167     shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
168     ~0**
169     ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
170     """
171     if not per_frame or frame_width <= 0 or frame_height <= 0:
172         return 0.0
173     aligned = 0
174     colocated = 0
175     for p in shuttle_points or []:
176         if not getattr(p, "visible", False):
177             continue
178         dets = per_frame.get(int(p.frame))
179         if dets is None: # shuttle frame wasn't person-sampled ? can't judge
180             continue
181         aligned += 1
182         if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
183               for _tid, box in dets):
184             colocated += 1
185     return colocated / aligned if aligned else 0.0
186

```

```

187
188 # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
189 # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
190 PERSON_FEATURE_NAMES = [
191     "players_in_court", "two_sided_motion", "mean_player_motion",
192     "in_court_ratio", "shuttle_player_colocation",
193 ]
194
195
196 def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
197                             frame_width: float, frame_height: float,
198                             court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
199                             net: Optional[Net] = None,
200                             min_players: int = 2) -> Dict[str, float]:
201     """Aggregate the per-window person features from per-frame detections + the in-
window
202 shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
203     counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
204     return {
205         "players_in_court": players_in_court(counts),
206         "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
207         "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
208         "in_court_ratio": in_court_ratio(counts, min_players),
209         "shuttle_player_colocation": shuttle_player_colocation(
210             per_frame, shuttle_points, frame_width, frame_height),
211     }
212

```

10. user (2026-06-13T19:09:39.565Z)

```

1     """WASB-based hybrid play segmenter.
2
3     Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
4     WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
5     badminton weights and runs today, so this is the live shuttle-motion substrate.
6
7     For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
8     candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
9     min_window_duration) are what get tuned against golden rally labels.
10
11     The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
12     lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
13     WASB runner and identity.
14     """
15     from typing import Any, Dict, Tuple
16
17     from backend.pipeline.segmenters.base import segmenter_registry
18     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
19     from backend.pipeline.detectors.base import DetectorRunner
20     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
21
22
23     class WasbHybridSegmenter(TrajectoryHybridSegmenter):
24         family = "wasb"
25         display_label = "WASB"
26
27         @property
28         def name(self) -> str:
29             return "wasb_hybrid"
30
31         @property
32         def description(self) -> str:

```



```

33         return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
34                 "weights) for precise candidate rallies + AI provider for semantic
boundaries.")
35
36     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
37         cfg = WasbConfig.from_indexing_cfg(idx_cfg)
38         return WasbRunner(cfg), {"weights_path": cfg.weights_path}
39
40
41     segmenter_registry.register(WasbHybridSegmenter)
42

```

11. user (2026-06-13T19:09:39.567Z)

```

1     """TrackNet-based hybrid play segmenter.
2
3     Phase 2 (candidate generation) uses a TrackNetV4 *shuttle* trajectory computed
4     in WSL2 (backend/pipeline/detectors/tracknet_runner.py, ADR-001). Rationale: a
5     tiny, fast, motion-blurred shuttle is exactly what general detectors miss; a
6     purpose-built trajectory model localises it far more precisely, giving cleaner
7     candidate rallies than person-velocity heuristics.
8
9     The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
10    lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
11    TrackNet runner and identity.
12    """
13    from typing import Any, Dict, Tuple
14
15    from backend.pipeline.segmenters.base import segmenter_registry
16    from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
17    from backend.pipeline.detectors.base import DetectorRunner
18    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
19
20
21    class TrackNetHybridSegmenter(TrajectoryHybridSegmenter):
22        family = "tracknet"
23        display_label = "TrackNet"
24
25        @property
26        def name(self) -> str:
27            return "tracknet_hybrid"
28
29        @property
30        def description(self) -> str:
31            return ("Two-stage pipeline: TrackNetV4 shuttle-trajectory detection (WSL2) for
precise "
32                    "candidate rallies + AI provider for high-precision semantic
boundaries.")
33
34    def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
35        cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
36        return TrackNetRunner(cfg), {
37            "weights_path": cfg.weights_path,
38            "queue_length": cfg.queue_length,
39        }
40
41
42    segmenter_registry.register(TrackNetHybridSegmenter)
43

```

12. assistant (2026-06-13T19:09:43.840Z)

Now let me verify the key NO-REGRESSION claims more precisely. Let me check the original (pre-change) version of the persistence block to confirm byte-identity, and check `add\_candidate\_segment`'s stats handling and `rally\_gate.gate\_windows` for any side effects.

13. user (2026-06-13T19:09:45.051Z)

```
t_start = time.time()
out_dir = os.path.join("output", self.family)
csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
if not csv_path:
    logger.error("%s produced no trajectory; aborting.", label)
    return []

points = runner.parse_trajectory_csv(csv_path)
logger.info("Parsed %d trajectory points (%d visible).",
            len(points), sum(1 for p in points if p.visible))

all_candidate_windows = runner.trajectory_to_action_windows(
    points, fps=fps, frame_width=frame_width, chunk_start=0.0,
    velocity_thresh=velocity_thresh, merge_gap=merge_gap,
    min_window_duration=min_window_duration, chunk_index=0,
)
logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
            label, time.time() - t_start, len(all_candidate_windows))

if log_candidates:
    for cw in all_candidate_windows:
        logger.info(f"[{label} rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
                    f"({cw['end'] - cw['start']:.1f}s) | "
                    f"peak_v={cw['peak_velocity']:.3f} "
                    f"mean_v={cw['mean_velocity']:.3f}")

# Persist raw candidates for the fine-tuning dataset (same table as YOLO).
candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
if store_candidates:
    for i, cw in enumerate(all_candidate_windows):
        stats = {
            "peak_velocity": cw["peak_velocity"],
            "mean_velocity": cw["mean_velocity"],
            "active_frame_count": cw["active_frame_count"],
            "track_id_count": cw["track_id_count"],
        }
        candidate_ids[i] = self.db.add_candidate_segment(
            video_id, detector=self.name,
            start_time=cw["start"], end_time=cw["end"],
            chunk_index=cw["chunk_index"], confidence=min(1.0, cw["peak_velocity"]),
            stats=stats, detection_params=detection_params,
        )

if not all_candidate_windows:
    return []

# --- Phase 3: AI provider handoff over padded candidate clips ---
handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
os.makedirs(handoff_dir, exist_ok=True)
logger.info("Phase 3: %s validation on %d candidate windows...",
            provider_name, len(all_candidate_windows))

def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
    w_start = max(0, window["start"] - handoff_padding)
    w_end = min(video_duration, window["end"] + handoff_padding)
    w_dur = w_end - w_start
    w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
    if not extract_video_chunk(video_path, w_start, w_dur, w_path, reencode=True):
```

```

        return []
    prompt = sport_profile.get_prompt(chunk_duration=w_dur)
    segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
                                         label=f"Handoff {wi+1}")

```

14. user (2026-06-13T19:09:45.612Z)

```

from backend.pipeline.segmenters.base import segmenter_registry, BasePlaySegmenter
from backend.pipeline.segmenters.motion import MotionPlaySegmenter
from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter

```

```

__all__ = [
    "segmenter_registry",
    "BasePlaySegmenter",
    "MotionPlaySegmenter",
    "GeminiPlaySegmenter",
    "HybridYoloSegmenter",
    "TrackNetHybridSegmenter",
    "WasbHybridSegmenter",
]

```

15. assistant (2026-06-13T19:09:49.462Z)

The persistence block is byte-identical: the new `window\_stats[i]` dict for the base class contains exactly the same 4 keys in the same order as the old inline `stats` dict. Now let me verify a few critical details: the `\_select\_for\_handoff` ordering of `wi` (futures iteration order), `add\_candidate\_segment` signature, and `rally\_gate.gate\_windows`.

16. user (2026-06-13T19:09:51.047Z)

```

240:     def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
241:                               end_time: float,
242:                               chunk_index: Optional[int] = None, confidence: Optional[float]
243:                               = None,
244:                               stats: Optional[Dict[str, Any]] = None,
245:                               detection_params: Optional[Dict[str, Any]] = None) ->
246: Optional[int]:
247:     """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
248:     detector-specific dicts stored as JSON. Returns the new candidate id, or None."""
249:     try:
250:         with self._connect() as conn:
251:             cursor = conn.cursor()
252:             cursor.execute(
253:                 """INSERT INTO candidate_segments
254:                 (video_id, detector, chunk_index, start_time, end_time, duration,
255:                 confidence, stats, detection_params)
256:                 VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
257:                 (video_id, detector, chunk_index, start_time, end_time, end_time -
258:                 start_time,
259:                 confidence, json.dumps(stats or {}), json.dumps(detection_params or
260:                 {}))
261:             )
262:             conn.commit()
263:             return cursor.lastrowid
264:     except Exception as e:
265:         logger.error(f"Failed to add candidate segment: {e}")
266:         return None
267:
268:     def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
269:     """Record that a candidate detection was confirmed as a given play_segment."""
270:     try:

```

```

266-         with self._connect() as conn:
267-             conn.cursor().execute(
268-                 "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id = ?",
269-                 (segment_id, candidate_id)
270-             )
271-             conn.commit()
272-         return True
273-     except Exception as e:
274-         logger.error(f"Failed to link candidate {candidate_id} to segment {segment_id}: {e}")
275-         return False

```

17. user (2026-06-13T19:09:52.151Z)

```

1: """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
29: class GatedWindow:
32:     crossings: int
69: def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
72:     crossings = 0
82:     crossings += 1
84:     return crossings
91: def gate_windows(points, windows: List[Interval], fps: float,
92:                 min_crossings: int = 2, deadband_frac: float = 0.06) -> List[GatedWindow]:
94:     keep decision (crossings >= min_crossings). Deadband is a fraction of the play
101:     c = count_net_crossings(wp, axis, net, deadband)
103:     out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
104:                          kept=c >= min_crossings))
109:     min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
112:     gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]

```

18. assistant (2026-06-13T19:09:55.978Z)

Now let me confirm two subtle correctness points: (1) the `stats` dict is JSON-serialized (so `window\_stats[i]` being a dict with the exact 4 keys produces identical JSON), and (2) `gate\_windows` with `min\_crossings=0` returns one `GatedWindow` per input window (so `gated[0]` is always valid for a single-window call). Let me read the full `gate\_windows` body.

19. user (2026-06-13T19:09:56.921Z)

```

85
86
87     def _points_in(points, fps: float, start: float, end: float):
88         return [p for p in points if start * fps <= p.frame <= end * fps]
89
90
91     def gate_windows(points, windows: List[Interval], fps: float,
92                     min_crossings: int = 2, deadband_frac: float = 0.06) ->
List[GatedWindow]:
93         """Annotate each (start,end) window with its shuttle net-crossing count and a
94         keep decision (crossings >= min_crossings). Deadband is a fraction of the play
95         axis span. Net/axis are estimated once over the whole trajectory (stable)."""
96         axis, net, span = estimate_play_axis(points)
97         deadband = deadband_frac * span
98         out: List[GatedWindow] = []
99         for (s, e) in windows:
100             wp = _points_in(points, fps, s, e)
101             c = count_net_crossings(wp, axis, net, deadband)
102             vis = sum(1 for p in wp if p.visible)
103             out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
104                               kept=c >= min_crossings))
105         return out
106
107
108     def apply_gate(points, windows: List[Interval], fps: float,

```

```

109         min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
110     """Convenience: return only the windows that pass the crossing gate."""
111     return [(g.start, g.end) for g in
112             gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
113

```

20. assistant (2026-06-13T19:10:01.442Z)

Confirmed: `gate\_windows` always returns exactly `len(windows)` entries, so `gated[0]` is safe when FusionSegmenter passes a single window. Now let me verify the empty-windows guard ordering carefully, and check the futures result-ordering concern. Let me also confirm `person\_detector.py` has the `Tuple`/`List` imports it now uses and `make\_tracker`/`detect\_and\_track` exist.

21. user (2026-06-13T19:10:03.098Z)

```

23:import logging
24:from typing import Any, Dict, List, Optional, Tuple
26:import cv2
27:import torch
63:    def lazy_load_model(self) -> None:
89:    def make_tracker(self, fps: float) -> Any:
98:    def detect_and_track(self, frame: Any, tracker: Any,

```

22. user (2026-06-13T19:10:04.276Z)

```

1     """Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.
2
3     Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):
4     the **person cue is a swappable producer**, not logic baked into one segmenter. Today
5     `HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-
identically);
6     next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and
7     later the learned classifier reads its persisted features. Keeping detection here (not
in
8     the segmenter) is what makes "add a cue = register a producer" true.
9
10    Behaviour is unchanged from the in-lined version ? only the home moved:
11    - `lazy_load_model`           ? load a permissively-licensed torchvision detector
12    - `make_tracker`             ? a fresh supervision ByteTrack per chunk
13    - `detect_and_track`         ? one frame ? confident persons with persistent IDs
14    - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows
15    with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
16    that feed the candidate `stats` JSON and the learned fusion features.
17
18    Heavy deps (torchvision, supervision) are imported lazily inside the methods so
importing
19    the segmenter registry never pulls torch on a machine that lacks it (and tests can mock
20    the seams). Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
21    (ADR-005; ultralytics' AGPL YOLO was dropped).
22    """
23    import logging
24    from typing import Any, Dict, List, Optional, Tuple
25
26    import cv2
27    import torch
28
29    from backend.utils.geometry import get_velocity_normalised
30

```

23. assistant (2026-06-13T19:10:12.057Z)

`person\_detector.py` imports `cv2` and `torch` at module top level (lines 26-27). This is the existing file behavior ? let me confirm the new `detections\_per\_frame` method doesn't change the

import posture, and verify whether the default fusion path truly avoids importing this module. The fusion segmenter imports `PersonDetector` lazily inside `\_person\_features` (line 92), so the default path never triggers it. Good.

Let me now check one subtle thing in the NO-REGRESSION claim about "empty-windows guard intact": in the new code, `window\_stats` is computed (line 203-206) and persistence loop runs (210-217) BEFORE the `if not all\_candidate\_windows: return []` guard (219-220). In the original, persistence also ran before the guard. Let me confirm both empty-list behaviors match and check whether `\_select\_for\_handoff` is reached when empty.

The guard at line 219 returns before `\_select\_for\_handoff` is called at line 225. With empty windows: `window\_stats` is `[]`, persistence loop iterates zero times, then guard returns `[]`. Identical to before.

Let me verify the futures result-ordering subtlety and confirm `Literal` is imported in models.py.

24. user (2026-06-13T19:10:13.606Z)

```
11:from typing import Any, List, Literal, Optional
15:Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
89:    substrate: Literal["wasb", "tracknet"] = "wasb"
103:    # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
104:    court_polygon: Optional[list[list[float]]] = None
107:    net_axis: Literal["x", "y"] = "y"
137:    # Optional debug knob: trim every video to the first N seconds before processing.
138:    debug_duration: Optional[float] = None
186:    position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
200:    allowed_video_root: Optional[str] = None
```

25. user (2026-06-13T19:10:14.681Z)

```
label=f"Handoff {wi+1}")
valid = []
for seg in segments:
    try:
        seg["start_time"] = float(seg["start_time"]) + w_start
        seg["end_time"] = float(seg["end_time"]) + w_start
        seg["_candidate_id"] = candidate_ids[wi]
        valid.append(seg)
    except (ValueError, TypeError, KeyError) as e:
        logger.warning("Dropping segment with unparseable timestamps: %s ? %s", seg,
e)

    try:
        os.remove(w_path)
    except OSError:
        pass
return valid

ai_segments: List[dict] = []
with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as executor:
    futures = [executor.submit(process_candidate_window, wi, cw)
                for wi, cw in enumerate(all_candidate_windows)]
    for future in futures:
```

26. assistant (2026-06-13T19:10:21.590Z)

All imports confirmed. Now let me verify the futures-ordering equivalence carefully.

Original: `futures = [executor.submit(process\_candidate\_window, wi, cw) for wi, cw in enumerate(all\_candidate\_windows)]` ? `wi` ranges 0..N-1, `cw = all\_candidate\_windows[wi]`.

New: `futures = [executor.submit(process_candidate_window, wi, all_candidate_windows[wi]) for wi

in handoff_indices]` where base `handoff_indices = list(range(len(windows)))` = `[0, 1, ..., N-1]`. So `wi` and the window passed are identical to the original for the base class. The result iteration order (`for future in futures`) is identical too. Byte-identical for the twins.

Now let me double-check one important detail: does `WasbConfig.from\_indexing\_cfg` read `velocity\_threshold` from `idx\_cfg["wasb"]` or `idx\_cfg["fusion"]`? The FusionSegmenter's `family = "fusion"`, so the engine reads `velocity\_thresh = idx\_cfg.get("fusion", {}).get("velocity\_threshold", 0.02)` (line 149). Let me verify the FusionConfig default (0.02) matches the WasbIndexConfig default to confirm the "seeds identically to wasb substrate" claim.

27. user (2026-06-13T19:10:22.262Z)

```
55         """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58         these knobs actually take effect ? previously this whole block was silently dropped
59         because nested config sections defaulted to extra='ignore'.
60         """
61         wsl_distro: str = "Ubuntu"
62         repo_dir: str = "~/models/WASB-SBDT"
63         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64         conda_env: str = "wasb"
65         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66         sport: str = "badminton"
67         wsl_stage_dir: str = "~/clips"
68         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69         velocity_threshold: float = 0.02
70         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
71         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
72         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
73         keep_frames: bool = False
74         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
75         min_free_gb: float = 0.0
76
```

28. assistant (2026-06-13T19:10:34.411Z)

Confirmed: `WasbIndexConfig.velocity\_threshold` default = 0.02 (line 69), matching `FusionConfig.velocity\_threshold` default = 0.02. Since the engine reads the windowing threshold from `idx\_cfg[self.family]["velocity\_threshold"]` and both families default to 0.02, the default FusionSegmenter seeds windows with the identical velocity threshold as `wasb\_hybrid`. All other windowing params (`merge\_gap`, `min\_window\_duration`) come from shared `idx\_cfg` keys, not family-scoped, so they're identical regardless.

Now let me verify one final correctness concern in the new `\_select\_for\_handoff` / Gate B logic ? the person Gate B reads `players\_in\_court` from stats, but that key is only present when the person cue is ON. The base default returns 0 windows-gated. Let me confirm there are no edge cases where the default FusionSegmenter could diverge. Default config: `cue\_flags={}` (person OFF), `min\_crossings=0`, `min\_players=0`.

In `\_enrich\_candidate\_stats` (fusion): base 4 keys + `net\_crossings` always added (line 78), person features skipped (person OFF). So default fusion persists 5 keys, not 4 ? but that's the *fusion* segmenter's own candidates (detector tag `fusion\_hybrid`), not wasb/tracknet. The NO-REGRESSION claim is about wasb/tracknet being unchanged, and about default fusion *seeding + handing off* identically (not persisting identically). Let me re-read the exact claim: "FusionSegmenter with DEFAULT config (person OFF, min_crossings=0) seeds + hands off identically to its wasb substrate." Persistence differs (extra `net\_crossings` key) but that's expected and not part of the claim. Seeding and handoff are what must match.

Let me confirm `\_select\_for\_handoff` for default fusion returns all indices: `min\_crossings=0` ? first `if` is `0 > 0` False, skipped; `person\_on=False` ? second `if` skipped. So `keep` = all

indices. Identical handoff to substrate. Confirmed.

Let me run the test suite for these modules to confirm nothing is broken, and check there are tests covering this.

29. user (2026-06-13T19:10:36.896Z)

```
Exit code 127
/usr/bin/bash: line 1: $null: ambiguous redirect
---untracked---
/usr/bin/bash: line 1: Select-String: command not found
```

30. user (2026-06-13T19:10:40.784Z)

```
backend\pipeline\detectors\fusion_features.py
backend\pipeline\segmenters\fusion_hybrid.py
tests\test_fusion_features.py
tests\__pycache__\test_fusion_features.cpython-313-pytest-9.0.3.pyc
tests\test_fusion_hybrid.py
tests\__pycache__\test_fusion_hybrid.cpython-313-pytest-9.0.3.pyc
backend\pipeline\detectors\__pycache__\fusion_features.cpython-313.pyc
backend\pipeline\segmenters\__pycache__\fusion_hybrid.cpython-313.pyc
```

31. user (2026-06-13T19:10:41.388Z)

```
M backend/config/models.py
M backend/pipeline/detectors/person_detector.py
M backend/pipeline/segmenters/__init__.py
M backend/pipeline/segmenters/trajectory_hybrid.py
M docs/README.md
M tests/test_config.py
M tests/test_trajectory_hybrid_equivalence.py
?? artifacts/
?? backend/pipeline/detectors/fusion_features.py
?? backend/pipeline/segmenters/fusion_hybrid.py
?? docs/QUALITY_ITERATIONS.md
?? scratch/HIGH_RISK_REMEDIATION_PLAN.md
?? scratch/PR1_BODY.md
?? scratch/PR1_HANDOFF.md
?? scratch/PR2_HANDOFF.md
?? scratch/PR2_deferred_hardening_status.patch
?? scratch/PR88_FIX.md
?? scratch/arial.ttf
?? scratch/debug_wsl.py
?? scratch/keep_awake.ps1
?? scratch/probe_cv2_count.py
?? scratch/probe_gemini.py
?? scratch/repro_drawtext_font.py
?? scratch/repro_movie_overlay.py
?? scratch/run_gx010125.py
?? scratch/stitch_min5.py
?? scratch/trim_video.py
?? tests/test_fusion_features.py
?? tests/test_fusion_hybrid.py
```

32. assistant (2026-06-13T19:10:44.255Z)

There's an existing equivalence test that was modified. Let me read it ? it directly tests the NO-REGRESSION claim ? plus the new fusion tests, to see what's actually covered.

33. user (2026-06-13T19:10:44.996Z)

```
1      """Equivalence contract for the trajectory-hybrid twins.
2
```



```

3     tracknet_hybrid and wasb_hybrid were copy-paste twins that drifted (the
4     shot_count parse); both are now thin subclasses of TrajectoryHybridSegmenter.
5     These tests run BOTH through the same scenarios and assert identical P3/P4
6     behavior, so any future divergence has to be introduced deliberately, in the
7     base, for both at once.
8     """
9     from unittest.mock import MagicMock, patch
10
11     import pytest
12
13     from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
14     from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
15     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
16
17     TWINS = [
18         (TrackNetHybridSegmenter, "tracknet", "tracknet_hybrid"),
19         (WasbHybridSegmenter, "wasb", "wasb_hybrid"),
20     ]
21
22
23     def _window(start=1.0, end=4.0):
24         return {
25             "start": start, "end": end, "active_frame_count": 30,
26             "peak_velocity": 0.4, "mean_velocity": 0.2, "track_id_count": 1,
27             "chunk_index": 0,
28         }
29
30
31     def _runner(windows=None):
32         runner = MagicMock()
33         runner.healthcheck.return_value = (True, "env ok")
34         runner.run_predict.return_value = "out.csv"
35         runner.parse_trajectory_csv.return_value = [MagicMock(visible=True)]
36         runner.trajectory_to_action_windows.return_value = windows if windows is not None
37     else [_window()]
38         return runner
39
40     def _run(cls, provider_segments, db=None):
41         """Run one segmenter class through the standard mocked pipeline."""
42         provider = MagicMock()
43         provider.analyze_video.return_value = (provider_segments, {})
44         db = db or MagicMock()
45         db.add_candidate_segment.return_value = 55
46         db.add_play_segment.return_value = 200
47
48         with patch.object(cls, "_build_runner", return_value=(_runner(), {"weights_path":
49 "w"})), \
50             patch.object(cls, "_probe_fps_width", return_value=(30.0, 1280.0)), \
51             patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
52 return_value=30.0), \
53             patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
54 return_value=True), \
55             patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry") as
56 preg, \
57             patch("os.environ.get", return_value="dummy_key"):
58             preg.get.return_value = provider
59             seg = cls(db, {"indexing": {}})
60             return db, seg.process_video("v1", "clip.mp4")
61
62 @pytest.mark.parametrize("cls, family, detector", TWINS)
63 def test_detector_identity_flows_into_db_rows(cls, family, detector):
64     db, res = _run(cls, [{"start_time": 0.5, "end_time": 3.0, "shuttle_exchange_count":
65 4}])

```

```

62     assert len(res) == 1
63     _, kwargs = db.add_candidate_segment.call_args
64     assert kwargs["detector"] == detector
65     assert kwargs["detection_params"]["detector"] == detector
66     # P4 metadata tags the family + model.
67     assert res[0]["metadata"]["detector"].startswith(f"{family}-hybrid-")
68     db.link_candidate_to_segment.assert_called_once_with(55, 200)
69
70
71     @pytest.mark.parametrize("cls,family,detector", TWINS)
72     @pytest.mark.parametrize("exchange,expected", [
73         (4, 4),          # provider-reported count wins
74         ("7", 7),        # numeric strings coerce
75         (None, 3),        # explicit null -> duration estimate (2.5s/0.8 -> 3 floor)
76         ("garbage", 3),   # unparseable -> same estimate
77     ])
78     def test_shot_count_behavior_is_identical(cls, family, detector, exchange, expected):
79         """The historical drift point: wasb_hybrid relied on int(None) raising into
80         the fallback while tracknet_hybrid short-circuited. One implementation now."""
81         seg_dict = {"start_time": 0.5, "end_time": 3.0, "shuttle_exchange_count": exchange}
82         _, res = _run(cls, [seg_dict])
83         assert res[0]["metadata"]["shot_count"] == expected
84
85
86     @pytest.mark.parametrize("cls,family,detector", TWINS)
87     def test_provider_offsets_applied_identically(cls, family, detector):
88         # Window [1,4] + 2.0s default padding -> clip starts at max(0, -1) = 0.0;
89         # provider-relative 0.5 stays 0.5 absolute here.
90         _, res = _run(cls, [{"start_time": 0.5, "end_time": 3.0}])
91         assert res[0]["start_time"] == 0.5
92         assert res[0]["end_time"] == 3.0
93
94
95     def test_base_is_not_registered():
96         from backend.pipeline.segmenters.base import segmenter_registry
97         available = segmenter_registry.list_available()
98         assert "tracknet_hybrid" in available
99         assert "wasb_hybrid" in available
100         # The base engine itself must not appear as a selectable segmenter.
101         assert not any(isinstance(cls, type) and cls is TrajectoryHybridSegmenter
102                        for cls in available.values())
103
104
105     @pytest.mark.parametrize("cls,family,detector", TWINS)
106     def test_persisted_stats_are_exactly_the_four_motion_keys(cls, family, detector):
107         """The fusion-P2 enrichment seam must leave the twins' persisted stats byte-
108         identical:
109         the base _enrich_candidate_stats is identity, so NO net_crossings / person keys leak
110         in."""
111         db, _res = _run(cls, [{"start_time": 0.5, "end_time": 3.0}])
112         _, kwargs = db.add_candidate_segment.call_args
113         assert set(kwargs["stats"]) == {
114             "peak_velocity", "mean_velocity", "active_frame_count", "track_id_count"}
115
116
117     @pytest.mark.parametrize("cls,family,detector", TWINS)
118     def test_base_fusion_hooks_are_identity(cls, family, detector):
119         """Directly pin the two fusion seams on the base: stats = the 4 motion keys
120         verbatim,
121         and every window is selected for handoff (no gating)."""
122         seg = cls(MagicMock(), {"indexing": {}})
123         cw = _window()
124         stats = seg._enrich_candidate_stats(cw, [], 30.0, 1280.0, "v.mp4", {})
125         assert stats == {"peak_velocity": 0.4, "mean_velocity": 0.2,
126                        "active_frame_count": 30, "track_id_count": 1}

```

```
124         assert seg._select_for_handoff([cw, cw, cw], [stats, stats, stats], {}) == [0, 1, 2]
125
```

34. assistant (2026-06-13T19:10:48.724Z)

The equivalence tests directly assert the byte-identity (``test_persisted_stats_are_exactly_the_four_motion_keys`, `test_base_fusion_hooks_are_identity``). Now let me read the new fusion_hybrid test to see what's covered for the default-config seeding/handoff claim, then run the relevant tests.

35. user (2026-06-13T19:10:49.386Z)

```
1      """FusionSegmenter tests (mocked runner/provider/db ? no GPU/WSL).
2
3      Proves: net_crossings is persisted; the person cue is OFF by default (no person-feature
4      keys, PersonDetector never built); the persisted candidate set is the full superset of
the
5      substrate windows; served Gate A drops sub-threshold windows from the AI handoff while
6      persistence keeps the superset; and the person cue, when enabled, adds the person
features.
7      """
8      from types import SimpleNamespace
9      from unittest.mock import MagicMock, patch
10
11      from backend.pipeline.detectors.fusion_features import PERSON_FEATURE_NAMES
12      from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
13
14      # Window A [1,4]s: shuttle alternates across the net every frame -> many crossings (a
rally).
15      # Window B [10,12]s: shuttle sits at the net level (held) -> 0 crossings. fps=30.
16      WIN_A = {"start": 1.0, "end": 4.0, "active_frame_count": 90, "peak_velocity": 0.4,
17              "mean_velocity": 0.2, "track_id_count": 2, "chunk_index": 0}
18      WIN_B = {"start": 10.0, "end": 12.0, "active_frame_count": 60, "peak_velocity": 0.1,
19              "mean_velocity": 0.05, "track_id_count": 1, "chunk_index": 0}
20
21
22      def _points():
23          pts = []
24          for f in range(30, 120):          # window A frames: y flips 100<->900 around the net
(~500)
25              pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=100.0 if f % 2 == 0
else 900.0))
26          for f in range(300, 360):          # window B frames: y == net level -> deadband -> no
crossings
27              pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=500.0))
28          return pts
29
30
31      def _runner():
32          r = MagicMock()
33          r.healthcheck.return_value = (True, "env ok")
34          r.run_predict.return_value = "out.csv"
35          r.parse_trajectory_csv.return_value = _points()
36          r.trajectory_to_action_windows.return_value = [dict(WIN_A), dict(WIN_B)]
37          return r
38
39
40      def _run(indexing=None, provider_segments=None):
41          provider = MagicMock()
42          provider.analyze_video.return_value = (provider_segments or [{"start_time": 0.5,
"end_time": 2.0}], {})
43          db = MagicMock()
44          db.add_candidate_segment.side_effect = [101, 102, 103, 104]
45          db.add_play_segment.return_value = 200
46
```

```

47         with patch.object(FusionSegmenter, "_build_runner", return_value=(_runner(),
{"weights_path": "w"})), \
48             patch.object(FusionSegmenter, "_probe_fps_width", return_value=(30.0, 1280.0)), \
49             patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
return_value=30.0), \
50             patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
return_value=True), \
51             patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry") as
preg, \
52             patch("os.environ.get", return_value="dummy_key"):
53             preg.get.return_value = provider
54             seg = FusionSegmenter(db, {"indexing": indexing or {}})
55             res = seg.process_video("v1", "clip.mp4")
56             return db, provider, res
57
58
59     def _stats_calls(db):
60         """Persisted stats dicts, in call order."""
61         return [kw["stats"] for _a, kw in db.add_candidate_segment.call_args_list]
62
63
64     def test_net_crossings_persisted_and_person_off_by_default():
65         db, _provider, _res = _run()
66         stats = _stats_calls(db)
67         assert len(stats) == 2                                # both windows persisted
(superset)
68         assert all("net_crossings" in s for s in stats)
69         # Window A (rally) crosses many times; window B (held) zero.
70         assert stats[0]["net_crossings"] >= 2
71         assert stats[1]["net_crossings"] == 0.0
72         # Person cue OFF by default -> NONE of the person features present.
73         assert all(not any(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
74         # The 4 base motion keys are still there.
75         assert all({"peak_velocity", "mean_velocity", "active_frame_count",
"track_id_count"} <= set(s)
76                     for s in stats)
77
78
79     def test_persisted_candidates_are_superset_of_substrate_windows():
80         db, _provider, _res = _run()
81         spans = [(kw["start_time"], kw["end_time"]) for _a, kw in
db.add_candidate_segment.call_args_list]
82         assert spans == [(1.0, 4.0), (10.0, 12.0)]            # exactly the substrate windows,
unfiltered
83
84
85     def test_default_hands_off_all_windows():
86         _db, provider, _res = _run()                            # min_crossings default 0 -> no
gating
87         assert provider.analyze_video.call_count == 2
88
89
90     def test_gate_a_drops_low_crossing_window_from_handoff_only():
91         db, provider, _res = _run(indexing={"fusion": {"min_crossings": 2}})
92         # Persistence is still the full superset (offline substrate intact)...
93         assert len(_stats_calls(db)) == 2
94         # ...but only the high-crossing window A reaches the AI handoff.
95         assert provider.analyze_video.call_count == 1
96
97
98     def test_person_cue_adds_features_when_enabled():
99         fake_pd = MagicMock()
100         fake_pd.detections_per_frame.return_value = {
101             "frames": {30: [(1, (10, 10, 30, 40))], 31: [(1, (12, 10, 32, 40))]},

```

```

102         "fps": 30.0, "frame_width": 1000.0, "frame_height": 1000.0,
103     }
104     with patch("backend.pipeline.detectors.person_detector.PersonDetector",
return_value=fake_pd):
105         db, _provider, _res = _run(indexing={"fusion": {"cue_flags": {"person": True}}})
106         stats = _stats_calls(db)
107         assert all(all(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
108         assert all("net_crossings" in s for s in stats)
109         fake_pd.detections_per_frame.assert_called()          # the person detector WAS used
110

```

36. assistant (2026-06-13T19:10:52.289Z)

The test coverage directly validates the default-config seeding and handoff claims. Let me run the relevant test files plus the full equivalence suite to confirm green.

37. user (2026-06-13T19:11:02.266Z)

```

..... [100%]
42 passed in 5.82s

```

38. assistant (2026-06-13T19:11:11.926Z)

All 42 tests pass. Now let me verify the one subtle behavioral concern I want to be certain about: the claim "window_stats computed even when store_candidates=False causes no behavioural change." For wasb/tracknet twins, `\_enrich\_candidate\_stats` is pure (returns a fresh dict from `cw` values, no side effects, no I/O). For FusionSegmenter, `\_enrich\_candidate\_stats` calls `rally\_gate.gate\_windows` (pure) and optionally person features. So computing `window\_stats` unconditionally when `store\_candidates=False` is purely wasted compute for the base twins ? no behavioral change since it has no side effects.

But there is a real concern for the FusionSegmenter: `window\_stats` is computed unconditionally (line 203-206), and for fusion that includes the GPU person path when the cue is on ? even if `store\_candidates=False`. Let me verify whether that's a behavioral change vs. expectations and whether it could break the `store\_candidates=False` no-op assumption. This is a fusion-specific consideration, not a twin regression. Let me confirm there's no path where the base twins do I/O in the hook.

The base `\_enrich\_candidate\_stats` (lines 89-94) only reads dict keys ? no I/O, no exceptions beyond KeyError if a key is missing (but those keys always exist on a window). Pure. Confirmed no behavioral change for twins when `store\_candidates=False`.

Let me do a final check on whether `idx\_cfg` passed to the hooks could ever be a non-dict (defensive), and confirm `WasbConfig.from\_indexing\_cfg` doesn't read the `fusion` section in a way that diverges.

39. user (2026-06-13T19:11:13.213Z)

```

45: def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
46:     w = (idx_cfg or {}).get("wasb", {}) or {}
47:     return cls(
48:         distro=w.get("wsl_distro", "Ubuntu"),
49:         repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
50:         conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
51:         conda_env=w.get("conda_env", "wasb"),
52:         weights_path=w.get("weights_path",
53:             "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
54:         sport=w.get("sport", "badminton"),
55:         wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
56:         timeout_sec=int(w.get("timeout_sec", 1800)),
57:         keep_frames=bool(w.get("keep_frames", False)),
58:         min_free_gb=float(w.get("min_free_gb", 0.0)),
59:     )

```

```

60-
61-
62-class WasbRunner(WslCommandMixin, DetectorRunner):
63-     def __init__(self, cfg: WasbConfig):

```

40. assistant (2026-06-13T19:11:21.502Z)

This confirms an important detail: ``WasbConfig.from_indexing_cfg`` reads from ``idx_cfg["wasb"]``, NOT ``idx_cfg["fusion"]``. So when the default `FusionSegmenter` builds its runner via ``_build_runner`` (line 67-68 of `fusion_hybrid.py`), it constructs ``WasbConfig.from_indexing_cfg(idx_cfg)`` reading ``idx_cfg["wasb"]`` ? the exact same runner config as ``wasb_hybrid``. The runner (WASB weights, distro, etc.) is identical to the wasb substrate.

So for the default `FusionSegmenter`:

- ****Runner****: reads ``idx_cfg["wasb"]`` ? identical WASB runner to ``wasb_hybrid``. ?
- ****Windowing velocity_threshold****: engine reads ``idx_cfg["fusion"]["velocity_threshold"]`` (default 0.02) = ``idx_cfg["wasb"]["velocity_threshold"]`` default (0.02). Identical *as long as the user hasn't tuned ``indexing.wasb.velocity_threshold``*. This is a real subtlety worth flagging.
- ****Seeding****: same runner + same threshold ? identical candidate windows. ?
- ****Handoff****: default Gate A/B OFF ? all windows handed off, same ``wi`` ordering. ?

I have enough to assess. The NO-REGRESSION claim holds for the twins (byte-identical) and for default-fusion seeding/handoff, with one genuine caveat about the `velocity_threshold` divergence if the wasb section is tuned. Let me compose the findings.

41. user (2026-06-13T19:11:50.234Z)

Structured output provided successfully

42. assistant (2026-06-13T19:12:02.498Z)

NO-REGRESSION claim VERIFIED. I read both new files, both edited twins, the pre-change HEAD of ``trajectory_hybrid.py``, ``add_candidate_segment``, ``rally_gate.gate_windows``, the ``WasbConfig`` builder, and ran the relevant test suite (42 passed).

All twin-facing claims hold byte-identically:

- ****Persisted stats unchanged**** ? base ``_enrich_candidate_stats`` returns the same 4 keys in the same order as the old inline dict; ``json.dumps(stats)`` produces an identical row.
- ****Handoff sends all windows**** ? base ``_select_for_handoff`` returns ``[0..N-1]``, so the new ``for wi in handoff_indices`` comprehension yields the same ``(wi, window)`` pairs and the same result-collection order as the old ``enumerate(...)``.
- ****Phase-3/Phase-4 unchanged**** ? the diff doesn't touch the candidate-processing closure or the DB population/link loop.
- ****Empty-windows guard intact**** ? ``if not all_candidate_windows: return []`` still sits before ``_select_for_handoff``; on empty input ``window_stats=[]`` and the function returns before any gating.
- ****store_candidates=False is a no-op for twins**** ? the unconditionally-computed ``window_stats`` uses a pure hook (dict reads only, no I/O/side effects).

Default ``FusionSegmenter`` also seeds + hands off identically to its wasb substrate: same WASB runner (read from ``indexing.wasb``), `velocity_threshold` default 0.02 == wasb default, and ``min_crossings=0``/``min_players=0``/person OFF keep every window in the handoff.

Two genuine low-severity caveats (neither breaks the claim as scoped):

1. Default fusion only matches a **tuned** wasb run if ``indexing.fusion.velocity_threshold`` is copied from a tuned ``indexing.wasb.velocity_threshold`` ? the runner reads ``indexing.wasb`` but the windowing threshold reads ``indexing.fusion`` (intentional, flagged in the docstring, but a config footgun).
2. Fusion-only: with the person cue ON, person features (GPU) are computed for every window regardless of ``store_candidates`` or Gate A pruning ? wasted work, not wrong output, and not a twin regression.

Relevant files: ``C:/Users/avidu/Projects/badminton-highlight-`

```
indexer/backend/pipeline/segmenters/trajectory_hybrid.py`, `C:/Users/avidu/Projects/badminton-  
highlight-indexer/backend/pipeline/segmenters/fusion_hybrid.py`,  
`C:/Users/avidu/Projects/badminton-highlight-  
indexer/tests/test_trajectory_hybrid_equivalence.py`, `C:/Users/avidu/Projects/badminton-  
highlight-indexer/tests/test_fusion_hybrid.py`.
```